MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión D del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_cruz_isaac
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/cruz_isaac/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas esa expresión.

Tu respuesta

```
New, recibe todas las clases que le ingreses al codigo, cada que abres una nueva clase, debes de meter el new, para justificarla, y que esta al compilarla, el compilador, pueda entender perfectamente lo que le estas queriendo decir
```

Corrección. La respuesta dice que new "recibe las clases" para que el compilador entienda el código, y eso no es correcto. new crea una instancia y ejecuta su constructor para inicializarla. La variable recibe una referencia a ese objeto.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué significa void en la cabecera de un método? Explica si ese método puede modificar un atributo.

Tu respuesta

```
Void, se usa para poder terminar un bucle, dependiendo de lo que se quiera efectuar, por lo que void, si se puede usar para modificar un atributo, siempre y cuando este sea un bucle, o un valor de repeticion
```

Corrección. Afirmar que el método puede modificar un atributo es válido, pero esa capacidad no depende de que haya un bucle. Además, void no sirve para terminar bucles. void indica que el método no devuelve ningún valor, aunque sí puede producir efectos como cambiar un atributo.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Para qué sirve this? Explica cómo distingue un atributo de un parámetro que tiene el mismo nombre.

Tu respuesta

```
This, es para poder clasificar ciertos parametros en el codigo, mientras que este puede diferenciar cosa como el parametro dependiendo del nombre, si previo a la variante this, se usa otra clase como el new
```

> **Corrección.** Se intuye que this ayuda a diferenciar elementos con el mismo nombre, pero la explicación es vaga y no tiene relación con new. this es una referencia al objeto actual: en this.temperatura = temperatura, la izquierda es el atributo y la derecha es el parámetro.

Pregunta 0.4 · Preguntas abiertas

Pregunta

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un int y cuando le pasas un arreglo?

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen JDK y JRE? Explica cuál necesitas para compilar un programa y por qué.

Tu respuesta

```
JDK es el que se necesita para compilar un pograma, y la diferencia es que en uno se puede codificar y el otro es exclusivo para poder compilar, por lo que JRE, solo sirve para codificar, para poder hacer tu codigo, y el otro es la extension, que ejecuta tu programa en caso de que hayas hecho bien todo desde un principio
```

> **Corrección.** Es correcto que el JDK se necesita para compilar. En cambio, el JRE no sirve para codificar: es el entorno de ejecución (la JVM y las bibliotecas). El JDK agrega herramientas de desarrollo como javac, y por eso es el que se necesita para compilar.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer base = 5;
Integer copia = base;
int[] aumentos = {1, 2, 3};
for (int aumento : aumentos) {
    base = base + aumento;
```

```
    System.out.println(base / 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Tu respuesta

una cadena de sucesion numerica, para un codigo de barras

Corrección. Se escribió una descripción en lugar de la salida del programa. La salida exacta era 3, 4, 5, D51 y 5.0, una por línea. base toma los valores 6, 8 y 11. copia sigue valiendo 5, así que "D" + 5 + 1 concatena "D51", y (double) da 5.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
if (true) { int voltaje = 12; }
System.out.println(voltaje);
```

Tu respuesta

Clasificación

Compila pero truena

Causa o salida

compila, debido a que el codigo esta bien escrito, pero va a tronar si se le escribe el valor de un voitaje diferente a 12

Corrección. La clasificación correcta era "no compila". La causa no tiene que ver con el valor 12: voltaje es una variable local del bloque del if y no existe fuera de él, aunque la condición sea true.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
StringBuilder ruta = new StringBuilder("Q");
ruta.append(2 + 3);
System.out.println(ruta);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
no compila, debido a que le hacen falta elementos, para que el codigo pueda correr de forma correcta
```

Corrección. El fragmento sí funciona e imprime Q5. No le falta ningún elemento: primero se evalúa 2 + 3 = 5 y después ese resultado se agrega al StringBuilder.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = null;
System.out.println(canales.length);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
no compila, porque le estas dando dos indicaciones al mismo tiempo, lo que hara que el compilador se confunda, ya que por un lado le estas pidiendo que mida, y por el otro rechazas cualquier valor numerico impreso
```

Corrección. La clasificación correcta era "compila pero truena". El código es válido para el compilador, pero canales es null y al leer length se lanza NullPointerException. La causa no son dos indicaciones simultáneas.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temperatura) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

Sin respuesta.

Justificaciones

Sin respuesta.

> **Corrección.** Sin respuesta.

R4 · Código con repetición

Pregunta

Devuelve el promedio entero de los valores pares de pulsos usando for-each. Si no hay pares, devuelve 0. El arreglo puede estar vacío, no es null y admite negativos. La suma cabe en int. No modifiques el arreglo.

```
static int promedioPares(int[] pulsos)
```

Ejemplos (argumentos → retorno):

{3, 8, 2, 5} → 5 · {1, 3, 5} → 0 · {-4, 3, 6} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
es un programa que se ejecuta mediante sumas, para despues el resultado de la suma dividirlo entre el numero de variantes
```

> **Corrección.** Se describió en prosa un promedio genérico (sumar y dividir), pero no se escribió el método. Esa descripción no filtra los pares ni trata el caso sin pares. Se esperaba static int promedioPares(int[] pulsos) con un for-each que sume y cuente solo los valores con v % 2 == 0, y que devuelva 0 cuando la cantidad es 0 o, si no, suma / cantidad.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la temperatura entera de una cámara de pruebas. Usa if / else-if / else y devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String estadoTermico(int temperatura)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 120	"INVALIDA"
0 a 29	"FRIA"
30 a 59	"TEMPLADA"
60 a 89	"CALIENTE"

90 a 120	"MUY CALIENTE"

Tu respuesta

```
aqui me pide un comparador, el comparador se encarga de checar los valores que ingrese en el programa, si estan dentro de cierto rango, este devolvera a la salida, cierto valor determinado, el cual ya esta tipificado en la tabla, solo puede ser dentro de los rangos antes vistos, por que si no truena, el programa esta hecho para que en caso de que este truene, te de un mensaje diciendo que el codigo es invalido
```

Corrección. La respuesta describe en prosa un "comparador" de rangos, pero no hay código ni se nombra ninguna de las categorías de la tabla. Se esperaba una cadena if / else-if / else que primero devuelva "INVALIDA" si temperatura < 0 || temperatura > 120 y luego "FRIA" (< 30), "TEMPLADA" (< 60), "CALIENTE" (< 90) o "MUY CALIENTE".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.